

AI-BASED EARLY DETECTION SYSTEM FOR DRY EYE DISEASE

PROJECT REPORT PHASE II

Submitted by

ANAND J L (CEM22CS014)
ASWAJITH S (CEM22CS024)
ASWIN GOPAKUMAR (CEM22CS025)
AU REVATHI (CEM22CS026)

to

the A P J Abdul Kalam Technological University



in partial fulfilment of the requirements for the award of the Degree

of

Bachelor of Technology

In

COMPUTER SCIENCE AND ENGINEERING



COLLEGE OF ENGINEERING MUTTATHARA
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
THIRUVANANTHAPURAM

APRIL 2026

AI-BASED EARLY DETECTION SYSTEM FOR DRY EYE DISEASE

PROJECT REPORT PHASE II

Submitted by

ANAND J L (CEM22CS014)
ASWAJITH S (CEM22CS024)
ASWIN GOPAKUMAR (CEM22CS025)
AU REVATHI (CEM22CS026)

to

the A P J Abdul Kalam Technological University



in partial fulfilment of the requirements for the award of the Degree

of

Bachelor of Technology

In

COMPUTER SCIENCE AND ENGINEERING



COLLEGE OF ENGINEERING MUTTATHARA
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
THIRUVANANTHAPURAM

APRIL 2026

VISION AND MISSION OF COLLEGE

VISION

To be a center of national excellence in technical education and research with strong ethical values.

MISSION

Create a continuous teaching and learning environment to serve the industry and society, Collaborate with other academic and research institutes to strengthen the education and research system. Work to inculcate high moral and ethical values amongst the students.

VISION AND MISSION OF DEPARTMENT

VISION

To generate competent professionals in Computer Science and Engineering by imparting quality education and there by facilitating cutting edge research and development to serve valuable resources for the industry and society.

MISSION

M1. Provide learning ambience through strong theoretical and practical background with an emphasis on software development.

M2. Establish Industry Interaction Programs to enhance the technical-know how at par with the current trends in the industry and to promote the entrepreneurship skills.

M3. Promote research-based projects in the emerging areas of Computer Science and Engineering.

M4. Empowering the youth in rural communities with computer education.

M5. Inculcating professional behaviour , leadership qualities, team spirit, skills on problem solving, critical thinking, and ethical responsibilities.

DECLARATION

We undersigned hereby declare that the project report “**AI-BASED EARLY DETECTION SYSTEM FOR DRY EYE DISEASE**”, submitted for fulfillment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by us under supervision of Dr. Praveen K Wilson, Assistant Professor, Dept of CSE, College of Engineering, Muttathara. This submission represents my ideas in my own words and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

Place: MUTTATHARA

DATE: 05-04-2026

ANAND J.L

ASWAJITH S

ASWIN GOPAKUMAR

A.U REVATHI

**DEPT. OF COMPUTER SCIENCE & ENGINEERING
COLLEGE OF ENGINEERING MUTTATHARA**

2025-2026



CERTIFICATE

This is to certify that the report entitled “ AI-BASED EARLY DETECTION SYSTEM FOR DRY EYE DISEASE ”, submitted by **ANAND J.L (CEM22CS014) , ASWAJITH S (CEM22CS024) , ASWIN GOPAKUMAR (CEM22CS025), A.U REVATHI (CEM22CS026)** to the APJ Abdul Kalam Technological University in fulfillment of the requirement for the award of B.Tech. degree in Computer Science and Engineering is a bonafide record of the project work carried out by us under our guidance and supervision. This report in any form has not been submitted to any other University or institute for any purpose

Project Guide

Dr. Praveen K Wilson
Assistant Professor
Dept. of CSE
College of Engineering Muttathara

Project Coordinator

Dr. Anoop S
Assistant Professor
Dept. of CSE
College of Engineering Muttathara

Head of the Department

Dr. Angel Viji K S
Associate Professor
Dept. of CSE
College of Engineering Muttathara

ACKNOWLEDGEMENT

We take this opportunity to express our deep sense of gratitude and sincere thanks to all who helped us to complete the work successfully. Our first and foremost thanks goes to God Almighty who showered in immense blessings on our effort. We wish to express our sincere thanks to **Dr. V Praseedalekshmi**, (Principal, College of Engineering Muttathara) for providing us with all the necessary facilities and support. We would like to express our sincere gratitude to **Dr. K S Angel Viji K S** (Head of the department) and **Dr. Anoop S** (Project Coordinator), for there support and co-operation. We wish to express our sincere gratitude towards our guide **Dr. Praveen K Wilson** for giving advices to work with the project and complete it successfully. We wish to express our sincere gratitude towards all the teaching and non- teaching staff members of our Department. Finally, We thank our parents, all our friends, near and dear ones who directly and indirectly contributed to the success of this work.

ANAND J.L
ASWAJITH
ASWIN GOPAKUMR
A.U REVATHI

ABSTRACT

This project presents an AI-Based Early Detection System for Dry Eye Disease (DED), addressing the rising prevalence of DED among individuals who spend long hours on screens in dry environments. Current diagnostic methods for DED are often expensive, require specialized clinical visits, and typically lead to detection only after symptoms, such as discomfort and blurred vision, have already occurred. Our system aims to overcome these limitations by providing a real-time, non-invasive, and cost-effective solution. Utilizing Machine Learning and Computer Vision technologies, the device continuously monitors a user's eye behaviour via a standard webcam to detect early signs of dry-eye conditions. The core methodology involves detecting the face and eyes using facial landmarks and calculating the Eye Aspect Ratio (EAR) in real time to measure eye openness. This data is used to analyse the user's blink rate and inter-blink intervals, identifying reduced blinking patterns that signify the onset of dry eyes. Crucially, the system features an Automated Health Intervention component: when dry-eye symptoms are confirmed, it automatically triggers a connected humidifier to increase local air moisture, providing immediate relief and preventing further discomfort. The system also maintains a detailed log of eye activity and humidifier usage for performance tracking and provides real-time status updates via a PC dashboard.

CONTENTS

ACKNOWLEDGEMENT	vi
ABSTRACT	vii
LIST OF FIGURES	xi
LIST OF TABLES.....	xii
CHAPTER 1	1
INTRODUCTION.....	1
1.1 OBJECTIVE.....	1
1.2 MOTIVATION	2
1.3 PROBLEM STATEMENT	2
CHAPTER 2	3
LITERATURE SURVEY	3
[16] An Improved Denoising Method for Eye Blink Detection Using Millimeter Wave Radar	3
[17] Title: Real-Time Multi-Person Eyeblink Detection in the Wild for Untrimmed Video	4
[18] Fusion of EEG and Eye Blink Analysis for Detection of Driver Fatigue	4
[19] Eye-LRCN: A Long-Term Recurrent Convolutional Network for Eye Blink Completeness Detection	5
[20] EEG-Based Eye Blink Detection and Interpretation for Human-Computer Interaction	5
[21] Robust Eye Blink Detection Using Dual Embedding Video Vision Transformer	6
2.1 RESEARCH GAP	6
2.2 COMPARISON OF EXISTING SYSTEMS	7
CHAPTER 3	8
METHODOLOGY.....	8
3.1 SYSTEM OBJECTIVES	8
3.2 METHODOLOGY	9

3.2.1 System Initialization.....	9
3.2.2 Face Detection and Frame Processing	9
3.2.3 Facial Landmark Detection	10
3.2.4 Eye Aspect Ratio (EAR) Calculation.....	10
3.2.5 Blink Detection Mechanism.....	10
3.2.6 Blink Rate Calculation (BPM).....	10
3.2.7 Dry Eye Classification	11
3.2.8 Automated Response system.....	11
3.2.9 Output Display and Logging	11
3.3 SYSTEM FLOW	11
CHAPTER 4	12
SYSTEM ARCHITECTURE AND DESIGN.....	12
4.1 SYSTEM ARCHITECTURE OVERVIEW	12
4.2 INPUT LAYER.....	13
4.3 PROCESSING LAYER	14
4.4 CONTROL AND HARWARE INTERFACE LAYER.....	15
4.5 DATA MANAGEMENT AND IOT LAYER	15
4.6 OUTPUT LAYER.....	15
4.8 SYSTEM COMPONENTS	16
CHAPTER 5	18
SYSTEM IMPLEMENTATION	18
5.1 DEVELOPMENT TOOLS AND TECHNOLOGIES	18
5.2 IMPLEMENTATION WORKFLOW.....	19
5.3 IMPLEMENTATION BASED ON SYSTEM LAYERS	21
5.3.1 Input Layer Implementation.....	21
5.3.2 Processing Layer Implementation.....	21
5.3.3 Control & Hardware Interface Layer Implementation.....	24

5.3.4 Data Management & IoT Layer Implementation.....	26
5.3.5 Output Layer Implementation	26
5.4 Performance Considerations	27
CHAPTER-6.....	28
RESULTS AND ANALYSIS.....	28
6.1 SYSTEM OUTPUT OBSERVATION	28
6.2 ACCURACY OF BLINK DETECTION	30
6.3 SYSTEM RESPONSE AND AUTOMATION	30
6.4 DATA HANDLING AND MONITORING	31
6.5 FUNCTIONAL ANALYSIS.....	31
6.6 PERFORMANCE ANALYSIS.....	31
6.7 RESULTANT OUTCOMES	32
6.8 LIMITATIONS	32
6.9 SUMMARY	32
CHAPTER-7.....	33
CONCLUSION AND FUTURE SCOPE	33
7.1 CONCLUSION	33
7.2 FUTURE SCOPE.....	34
REFERENCES.....	35
APPENDIX	37

LIST OF FIGURES

Figure 3.3 Simplified System Flow of Dry Eye Detection	11
Figure 4.1. System Architecture	13
Figure 5.2 Workflow of System	20
Figure 5.3.2.1 68-Point Facial Landmark Detection Model	22
Figure 5.3.2.2 Open vs Closed Eye EAR Representation	23
Figure 5.3.3.1 IoT Smart Humidifier Control System	24
Figure 5.3.3.2 Physical Humidifier Setup (i)	25
Figure 5.3.3.2 Physical Humidifier Setup (ii)	25
Figure 5.3.5 Blink Rate Trend Analysis with Health Thresholds	27
Figure 6.1.1: User Interface of the Dry Eye Monitoring System	28
Figure 6.1.2: Patient History Interface (i)	28
Figure 6.1.2: Patient History Interface (ii)	29
Figure 6.1.3: BPM & Dry eye events graph	30
Figure E.1.1 Smart Identity Lock & Data Protection Module Code	39
Figure E.1.2 Smart Identity Lock & Data Protection Module Code	39
Figure E.2 The Computer Vision & AI Diagnostic Pipeline Code	40
Figure E.3.1 Hardware Automation & Cloud IoT Synchronization Code	41
Figure E.3.2 Hardware Automation & Cloud IoT Synchronization Code	41
Figure E.4.1 Real-Time Asynchronous Web Dashboard Engine Code	42
Figure E.4.2 Real-Time Asynchronous Web Dashboard Engine Code	42
Figure E.4.3 Real-Time Asynchronous Web Dashboard Engine Code	43
Figure E.4.4 Real-Time Asynchronous Web Dashboard Engine Code	43
Figure E.5.1 Historical Patient Data Visualization Engine Code	44
Figure E.5.2 Historical Patient Data Visualization Engine Code	44

LIST OF TABLES

Table 2.2 Comparative Analysis of Existing Eye Blink Detection Techniques	7
Table 4.8 System Components and Functional Description of Dry Eye Monitoring System	16
Table 6.5 Functional Modules and Implementation Status.....	31
Table 6.6 Performance Analysis of the Proposed System	31

CHAPTER 1

INTRODUCTION

Dry Eye Disease (DED) is a growing concern, primarily driven by prolonged digital screen use and exposure to low-humidity environments, which can lead to significant discomfort, blurred vision, and potential long-term damage if left untreated. The challenge lies in the fact that current DED diagnosis relies on expensive, specialized clinical equipment and must be done by trained doctors, making continuous, early monitoring impractical for most users. To address this critical gap, this project proposes the development of an **AI-Based Early Detection System for Dry Eye Disease** that integrates Machine Learning (ML) and Computer Vision to offer a non-invasive, cost-effective, and portable solution. The system's primary goal is to continuously monitor a user's blink rate in real time via a standard webcam to detect early dry-eye symptoms and automatically trigger an environmental intervention, such as activating a humidifier, to provide instant and automated relief.

1.1 OBJECTIVE

- **Real-Time Eye Behaviour Monitoring:** To continuously monitor the user's blink pattern via a webcam using computer vision techniques for real-time detection of eye behavior.
- **Dry-Eye Detection through Pattern Analysis:** To analyze the blink rate and inter-blink intervals in order to effectively detect reduced blinking patterns that are associated with dry eye conditions.
- **Automated Environmental Intervention:** To automatically trigger and control a connected humidifier when dry-eye symptoms are detected, thereby actively improving the local humidity.
- **Logging, Feedback, and Safety Control:** To provide real-time alerts and status updates through a PC dashboard, and to maintain a detailed log of blink rates, dry-eye detections, and humidifier usage for performance tracking, while also implementing safety measures like time limits and cooldown periods.

1.2 MOTIVATION

The motivation for this project is rooted in the significant public health challenge posed by the increasing prevalence of Dry Eye Disease (DED), which is closely linked to extended use of digital screens and low-humidity environments. The discomfort, blurred vision, and potential for permanent eye damage caused by DED underscore the need for effective intervention. Critically, existing solutions are often inadequate: current methods are expensive, require specialized equipment, and necessitate clinic visits, making them inconvenient and unsuitable for regular, proactive monitoring. This results in late detection—problems are noticed only after symptoms have already appeared. Therefore, the core motivation is to pioneer a system that is affordable, portable, and easy-to-use, capable of real-time early detection of dry-eye symptoms via a simple webcam, and most importantly, provides automatic prevention through self-triggered humidification, shifting eye care from reactive treatment to proactive, continuous management.

1.3 PROBLEM STATEMENT

Dry Eye Disease (DED) is an increasingly common public health issue, exacerbated by prolonged use of digital screens and exposure to dry working environments, which often leads to discomfort, blurred vision, and, in severe cases, permanent eye damage. A major challenge is that many users fail to notice the problem until symptoms worsen. Current diagnostic methods for DED are inaccessible for regular use, as they are expensive, require specialized medical machines, and must be performed by trained doctors in clinics. This makes it difficult for individuals to check their eyes regularly. Furthermore, existing systems often fail to accurately detect partial or incomplete blinks, which are a key cause of dry eye, and they lack automation, requiring manual intervention for prevention or treatment. This lack of an affordable and easy-to-use continuous monitoring system prevents early detection and proactive intervention. Therefore, the central problem this project addresses is the absence of a small, portable, and cost-effective device that can both detect DED early in real time using common hardware (like a webcam) and automatically provide quick relief by increasing air moisture to enhance eye comfort. Developing a system capable of robust, real-time detection and automated environmental control is essential to providing convenience and preventing the progression of DED.

CHAPTER 2

LITERATURE SURVEY

The literature shows that Eye Aspect Ratio (EAR) methods are efficient for real-time blink detection but struggle with incomplete blinks and real-world conditions like lighting and head movement. While deep learning models improve accuracy, they depend on costly clinical devices. This highlights the need for an affordable, non-invasive system that can accurately analyze blink frequency and completeness using consumer-grade hardware.

[16] An Improved Denoising Method for Eye Blink Detection Using Millimeter Wave Radar

Authors: Yuhong Shu, Yong Wang, Xiaobo Yang, Zengshan Tian **2022**

This paper proposes a radar-based approach for detecting eye blinks using millimeter wave signals instead of cameras. It captures small motion changes caused by eyelid movement, making it useful in situations where cameras may fail. This approach also improves privacy since no visual images are recorded.

The main contribution is an improved denoising algorithm that removes noise from radar signals, allowing more accurate detection of blink patterns. This helps maintain performance even in low-light or visually obstructed environments. The method also enhances signal clarity, which is important for detecting subtle blink movements. The method is reliable and works well in challenging conditions, but it requires specialized radar hardware. This makes it less suitable for low-cost, consumer-level applications compared to webcam-based systems. Therefore, it is more applicable in industrial or research settings.

Additionally, the study highlights the potential of non-visual sensing techniques in human behavior analysis. It opens opportunities for future research in integrating radar with other sensing methods to improve accuracy. However, further work is needed to make such systems more affordable and widely accessible.

[17] Title: Real-Time Multi-Person Eyeblick Detection in the Wild for Untrimmed Video

Authors: Wenzheng Zeng, Yang Xiao, Sicheng Wei, Jinfang Gan, Xintao Zhang, Zhiguo Cao 2023

This research focuses on detecting eye blinks in real-world videos containing multiple people. The system is designed to work in unconstrained environments with varying lighting, head movements, and background noise. This makes it suitable for real-life scenarios rather than controlled environments.

The model uses both spatial and temporal features to track and detect blinks across multiple faces simultaneously. It also works on untrimmed videos, meaning it does not require pre-processed input. This reduces the need for manual data preparation and improves usability.

The approach improves robustness and scalability for real-world applications such as surveillance and monitoring. However, the system can be computationally demanding for real-time use on low-power devices. Optimization is required for deployment on edge devices.

[18] Fusion of EEG and Eye Blink Analysis for Detection of Driver Fatigue

Authors: Mohammad Shahbakhti, Matin Beiramvand, Erfan Nasiri, Somayeh Mohammadi Far Wei Chen, Jordi Sole-Casals 2023

This paper introduces a multimodal system that combines EEG signals with eye blink analysis to detect driver fatigue. By using both physiological and visual data, the system improves detection accuracy significantly. It reduces the chances of false detection compared to single-source methods.

Machine learning techniques are used to fuse EEG features with blink patterns, enabling more reliable identification of fatigue levels. This reduces the limitations of camera-only systems, especially under poor lighting conditions. The system can detect early signs of fatigue more effectively.

Although the results are highly accurate, the need for EEG sensors makes the system complex and expensive. This limits its use in everyday consumer applications and favours controlled environments. It is mainly suitable for research or high-end safety systems.

[19] Eye-LRCN: A Long-Term Recurrent Convolutional Network for Eye Blink Completeness Detection

Authors: Gonzalo de la Cruz, Madalena Lira, Oscar Luaces, Beatriz Remeseiro 2024

This paper presents the Eye-LRCN model, which combines CNN and LSTM networks to analyse both spatial and temporal features of eye movements. The focus is on detecting blink completeness, which is important for Dry Eye Disease diagnosis. This makes it more clinically relevant than basic blink detection.

The model processes sequences of eye images to understand blink behaviour over time. It can effectively distinguish between complete and incomplete blinks, improving diagnostic accuracy compared to traditional methods like EAR. The temporal modelling helps capture subtle blink patterns.

While the model achieves high performance, it requires large datasets and higher computational power. This makes real-time implementation on simple devices more challenging without optimization. Efficient model design is needed for practical applications.

[20] EEG-Based Eye Blink Detection and Interpretation for Human-Computer Interaction

Authors: Deepu S.S, Chithra P, Praveen Panicker, P. S. Subodh 2024

This study explores the use of EEG signals to detect eye blinks and use them as input for human-computer interaction systems. It allows users to control devices using blink patterns, which is helpful for assistive technologies. This is especially useful for users with limited physical mobility.

The system uses machine learning models to classify EEG signals related to different types of blinks. It achieves good accuracy and shows potential for applications in accessibility and communication systems. The method also enables hands-free interaction. However, the requirement of EEG equipment increases cost and setup complexity. This makes it less practical for everyday use compared to vision-based blink detection systems. Simpler alternatives are preferred for consumer applications.

[21] Robust Eye Blink Detection Using Dual Embedding Video Vision Transformer

Authors: Jeongmin Hong, Joseph Shin, Juhee Choi, Minsam Ko 2024

This paper proposes a Vision Transformer-based model for eye blink detection using dual embedding techniques. It captures both spatial and temporal information from video sequences more effectively than traditional models. This improves the overall understanding of blink behavior.

The approach improves robustness against variations such as head pose, lighting conditions, and occlusions. The attention mechanism in transformers helps focus on important features for accurate blink detection. It performs well even in challenging real-world scenarios.

Although the model provides high accuracy, it is computationally expensive and requires powerful hardware. This limits its use in real-time applications on low-cost consumer devices. Model optimization is necessary for practical deployment.

2.1 RESEARCH GAP

Existing eye blink detection methods use advanced techniques like radar, EEG, and deep learning. While they improve accuracy and work in challenging conditions, they rely on expensive hardware or high computational power, making them unsuitable for low-cost, real-time applications on basic devices.

Some approaches also focus on multi-person detection or fatigue analysis, but they are often complex and not optimized for continuous, real-time monitoring of eye health conditions like Dry Eye Disease.

Therefore, there is a need for a cost-effective, lightweight system that works in real time using simple hardware like webcams, while also providing meaningful insights such as dry eye classification and automated response.

2.2 COMPARISON OF EXISTING SYSTEMS

Paper Name	Technique Used	Key Features	Advantages	Limitations
Radar-Based Blink Detection (2022)	mmWave Radar + Denoising	Camera-free blink detection, noise removal	Works in low light, privacy-safe	Expensive hardware
Multi-Person Blink Detection (2023)	Spatial + Temporal Analysis	Detects multiple faces in real-world videos	Robust, scalable	High computation
EEG + Blink for Fatigue (2023)	EEG + ML Fusion	Combines brain signals and blink data	High accuracy	Costly, complex setup
Eye-LRCN Model (2024)	CNN + LSTM	Detects blink completeness	Accurate, clinically useful	Needs large data, high compute
EEG Blink for HCI (2024)	EEG + ML	Blink-based device control	Hands-free use	Expensive, complex
Vision Transformer Blink (2024)	Vision Transformer	Attention-based blink detection	High accuracy, robust	High computational cost

Table 2.2 Comparative Analysis of Existing Eye Blink Detection Techniques

CHAPTER 3

METHODOLOGY

This chapter presents the overall methodology adopted for the development of the AI-Based Early Detection System for Dry Eye Disease (DED). The methodology focuses on integrating Computer Vision and Machine Learning techniques to create a real-time monitoring and automated response system. The system is designed to continuously track eye behaviour, analyse blinking patterns, and provide immediate intervention when abnormal conditions are detected.

The development follows a structured approach including system initialization, face detection, eye analysis, blink detection, classification, and automated response. The methodology ensures accuracy, efficiency, and real-time performance while maintaining user-friendliness and portability.

3.1 SYSTEM OBJECTIVES

The primary objective of the system is to detect early signs of Dry Eye Disease through continuous monitoring of eye behavior using a webcam and to provide automated corrective measures that help maintain optimal eye health and comfort.

The system is designed to operate in real time, ensuring that users receive immediate feedback and timely interventions. By leveraging computer vision techniques and intelligent analysis, the system aims to reduce the risk of prolonged eye strain and prevent the progression of dry eye conditions, especially for individuals who spend extended hours in front of digital screens.

The specific objectives of the system are as follows:

- The system captures real-time video input continuously using a webcam to monitor the user's facial and eye movements.
- The system detects and recognizes the user's face accurately under varying lighting and environmental conditions to ensure consistent tracking.
- The system identifies facial landmarks precisely to extract the eye region and calculates the Eye Aspect Ratio (EAR) to determine the state of the eyes in each frame.

- The system detects and counts eye blinks using frame-based analysis to monitor blinking patterns over time.
- The system computes the blink rate in terms of blinks per minute (BPM) to assess eye activity levels.
- The system classifies the user's eye condition into categories such as Healthy, Moderate, or Severe based on analyzed blink patterns and thresholds.
- The system triggers an automated environmental response, such as activating a humidifier, when signs of dry eye are detected.
- The system maintains logs of user eye activity and system responses for future analysis and tracking.
- The system provides real-time feedback and alerts to the user while ensuring high accuracy, efficiency, and reliability through optimized algorithms.
- The system is designed to be user-friendly, non-intrusive, scalable, and adaptable for future health monitoring integration.

3.2 METHODOLOGY

3.2.1 System Initialization

- The system begins by preparing the environment for real-time execution and importing required libraries such as OpenCV, dlib, and NumPy.
- A pre-trained facial landmark detection model is loaded and the face detection module is initialized to enable detection capabilities.
- Necessary directories and data structures are created to manage processing and storage.
- The webcam is activated for continuous video capture, and variables for blink detection and classification are initialized to prepare the system for processing.

3.2.2 Face Detection and Frame Processing

- Once initialized, video frames are continuously captured and processed in real time.
- Frames are converted into grayscale to improve computational efficiency.
- Face detection is performed using a frontal face detector.
- The detected face region is extracted for further analysis, isolating the region of interest.

3.2.3 Facial Landmark Detection

- After detecting the face, key facial features are identified using a 68-point facial landmark model.
- The coordinates of important facial features are extracted from the model.
- Eye regions are identified using specific landmark points (37–42 for left eye and 43–48 for right eye).
- These landmark points are converted into structured arrays for precise localization and further computation.

3.2.4 Eye Aspect Ratio (EAR) Calculation

- The Eye Aspect Ratio (EAR) is calculated to determine whether the eye is open or closed.
- Vertical distances between the eyelids and horizontal distance between eye corners are measured.
- Using these values, the EAR is computed based on the formula.
- A predefined threshold classifies the eye state as open or closed, enabling efficient real-time detection.

3.2.5 Blink Detection Mechanism

- Blink detection is performed by analyzing EAR values over consecutive frames.
- EAR values are continuously monitored, and frames below the threshold are counted.
- A blink is confirmed only if the eye remains closed for a minimum number of consecutive frames, after which the blink count is incremented.
- The frame counter is reset after each detection to improve accuracy and reduce noise.

3.2.6 Blink Rate Calculation (BPM)

- The system calculates blink rate in terms of blinks per minute (BPM).
- The total number of blinks is recorded over time.
- BPM is computed based on the duration of observation.
- This value is continuously updated and used as an important parameter for detecting dry eye conditions.

3.2.7 Dry Eye Classification

- The system classifies the eye condition based on blink rate (BPM).
- Healthy condition: $\text{BPM} \geq 20$, indicating normal blinking behavior.
- Moderate condition: $10 \leq \text{BPM} < 20$, indicating reduced blinking.
- Severe condition: $\text{BPM} < 10$, indicating high risk of dry eye, with real-time updates.

3.2.8 Automated Response System

- The system provides automated responses based on detected eye condition and continuously monitors classification results.
- When a severe condition is detected, a humidifier is automatically activated.
- Alerts are generated to notify the user about the condition.
- The system controls responses using limits to prevent excessive or unnecessary usage.

3.2.9 Output Display and Logging

- The system provides real-time feedback and maintains logs for analysis.
- It displays EAR, blink count, BPM, and eye condition status on the screen.
- Logs store blink patterns, classification results, and system responses.
- This enables both real-time monitoring and future analysis of user data.

3.3 SYSTEM FLOW



Figure 3.3 Simplified System Flow of Dry Eye Detection

CHAPTER 4

SYSTEM ARCHITECTURE AND DESIGN

This chapter explains the architecture and design of the AI-Based Early Detection System for Dry Eye Disease (DED). The system is designed to ensure efficient data flow, modularity, and real-time performance. The architecture integrates computer vision techniques with a structured processing pipeline to deliver accurate detection and automated response. A well-defined architecture helps in organizing system components, improving scalability, and ensuring ease of maintenance. The design focuses on providing a seamless user experience with efficient processing and reliable output.

4.1 SYSTEM ARCHITECTURE OVERVIEW

The system is designed using a layered architecture that divides its functionality into Five primary layers:

- Input Layer.
- Processing Layer.
- Control & Hardware Interface Layer.
- Data Management & IoT Layer.
- Output Layer.

Each layer is responsible for specific operations and communicates seamlessly with the others to ensure smooth and efficient system performance. This structured and modular approach enhances flexibility, allowing independent development and modification of individual components without affecting the entire system. Additionally, it improves maintainability and scalability while ensuring optimal performance across different operating conditions.

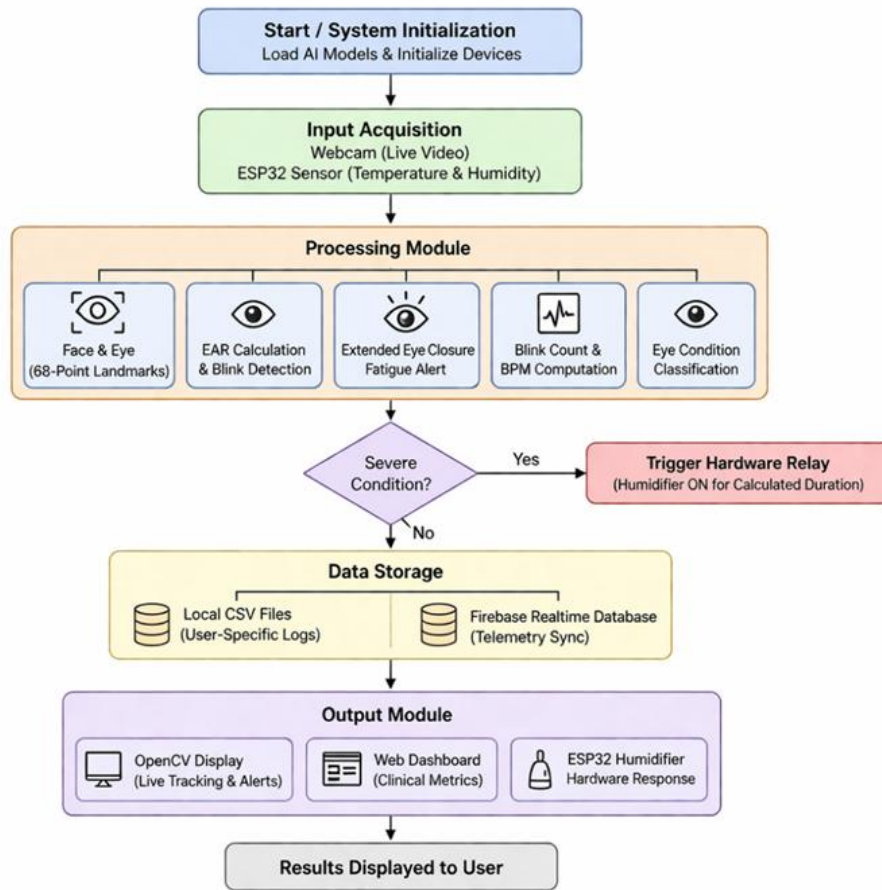


Figure 4.1. System Architecture

4.2 INPUT LAYER

The Input Layer is responsible for collecting real-time data from the user and environment

Functions:

- Captures live video stream using webcam
- Accepts user input such as registration and face positioning
- Receives environmental data (temperature and humidity) from sensors via ESP32

Importance:

This layer acts as the entry point of the system, ensuring continuous and accurate data acquisition required for further processing.

4.3 PROCESSING LAYER

The Processing Layer is the core computational unit where all analysis takes place.

Modules and Functions:

- Face detection and recognition:

This step identifies and verifies the user's face from the live video feed. The system detects a face in each frame and matches it with stored facial data to ensure only registered users are tracked, maintaining security and personalized monitoring.

- Facial landmark detection:

The system locates key points on the detected face, especially around the eyes, using a landmark model. These points help in accurately isolating the eye region, which is essential for further analysis.

- Eye Aspect Ratio (EAR) calculation:

EAR is calculated using distances between eye landmarks to determine whether the eye is open or closed. The value decreases significantly when the eye closes, making it a reliable indicator of eye state.

- Blink detection:

Blink detection is done by monitoring changes in EAR over consecutive frames. If the EAR drops below a threshold for a short duration, it is counted as a blink, ensuring accurate detection while reducing noise.

- Blinks Per Minute (BPM) calculation:

The system calculates BPM by counting the number of blinks over time and converting it into a per-minute value. This provides a measurable indicator of blinking behavior.

- Dry eye classification:

Based on the BPM value, the system classifies the eye condition into categories like healthy, moderate, or severe. This helps in understanding the user's eye health in a simple and meaningful way.

4.4 CONTROL AND HARWARE INTERFACE LAYER

This layer acts as the bridge between software and physical hardware components.

Functions:

- Communicates with ESP32 microcontroller via serial communication
- Controls humidifier using relay module
- Implements time-proportional control for spray duration
- Applies safety mechanisms such as cooldown period

Importance:

This layer enables automated physical response, ensuring that detected dry eye conditions trigger real-world corrective actions safely and efficiently.

4.5 DATA MANAGEMENT AND IOT LAYER

This layer is responsible for managing data storage, integrating with cloud platforms, and enabling remote monitoring of the system. It ensures that all session data is properly stored, securely transmitted to the cloud, and made accessible for real-time tracking and future analysis.

Functions:

- The system stores session data in CSV files for organized record keeping and analysis.
- It also sends real-time data to the Firebase Realtime Database for remote monitoring and cloud storage
- Manages key parameters:
 - Blink count
 - BPM
 - Temperature and humidity
 - System status
- Enables remote monitoring through web dashboard
- Maintains historical records for long-term analysis

4.6 OUTPUT LAYER

The Output Layer presents processed information and system responses to the user.

- The system displays real-time BPM and classifies the user's eye condition as Healthy, Moderate, or Severe based on analysis.

- It provides alerts for dry eye conditions and triggers high-priority warnings for extended eye closures such as fatigue or microsleep.
- The system generates graphical visualizations of blink patterns to help in understanding trends and variations.
- It displays system status, feedback, and two-way real-time telemetry indicating the humidifier status (ON/OFF) for hardware confirmation.
- The system tracks "Dry Eye Events" using strict thresholds and provides a historical dashboard for reviewing past session data and user-specific trends.

4.8 SYSTEM COMPONENTS

Component	Category	Description	Function	Technical Details	Importance
Camera Module	Input Unit	A camera used to capture real-time video of the user's face and eye region.	Captures continuous video frames for blink detection.	Frame rate (fps) and resolution affect detection accuracy.	Primary source of input; determines quality of detection.
User Interface (Face Positioning)	Input Unit	Refers to user positioning in front of the camera.	Ensures proper alignment of face and eyes for detection.	Stable positioning improves detection reliability.	Reduces errors due to improper input.
Computer Processing Device	Processing Unit	A system (PC/Laptop) that runs the detection program.	Executes algorithms and processes input data.	Requires sufficient processing power for real-time analysis.	Acts as the brain of the system.
Image Processing Module	Processing Unit	Software module that processes captured frames.	Detects face and extracts eye region.	Uses computer vision techniques (e.g., OpenCV).	Prepares data for blink detection.
Blink Detection Module	Processing Unit	Core module that identifies eye blinks.	Detects eye open/closed states and counts blinks.	Uses algorithms like Eye Aspect Ratio (EAR) or thresholding.	Key component for accurate blink detection.
BPM Calculation Module	Processing Unit	Module that calculates blinks per minute.	Computes BPM using blink count over time.	$BPM = \frac{\text{Number of Blinks}}{\text{Time (in minutes)}}$	Provides main output parameter.

Data Logging Module	Storage Unit	Module that records blink data.	Stores BPM values with timestamps.	Uses structured storage formats.	Enables historical analysis and tracking.
File Storage System	Storage Unit	Local storage for saving data and graphs.	Stores CSV/text files and graph images.	Organized in folders (e.g., user graphs).	Ensures data persistence and easy retrieval.
Display Module	Output Unit	Interface to show system results.	Displays real-time BPM values.	Can be console output or GUI display.	Provides immediate feedback to the user.
Graph Visualization Module	Output Unit	Module for plotting blink data.	Generates graphs representing BPM vs time.	Implemented using libraries like Matplotlib.	Helps in analyzing blink patterns visually.
Alert / Analysis Module	Output Unit	Module for detecting abnormal blinks.	Identifies fatigue or irregular blinking.	Based on threshold values.	Enhances system functionality and safety.
Power Supply Unit	Supporting Unit	Provides electrical power to the system components.	Supplies required voltage and current.	Can be USB power or battery; regulated	Ensures stable operation of the system.
ESP32	Edge Processing Unit	Hardware bridge between PC and sensors	Receives Python commands and reads DHT sensors.	Uses two-way serial communication at 115200 baud.	Enables physical IoT control.
Relay Module	Output Unit	Electronic switch.	Controls the high-power humidifier	Triggered by ESP32 digital pins	Safely isolates PC from humidifier voltage.
DHT11 Sensor	Input Unit	Environmental sensor	Captures ambient room temperature and humidity	Sends formatted data to ESP32	Provides live environmental context for the dashboard.

Table 4.8 System Components and Functional Description of Dry Eye Monitoring System

CHAPTER 5

SYSTEM IMPLEMENTATION

This chapter presents the implementation of the proposed Dry Eye Detection and Automated Response System. It converts the system design into a fully functional model by integrating computer vision, real-time processing, IoT communication, and cloud-based data management for accurate monitoring and intelligent response. The implementation follows a layered approach, where each layer performs specific tasks while continuously interacting with others. This structure ensures smooth operation, efficient performance, and easy scalability. The system operates in real time, enabling continuous detection, analysis, data logging, and automated hardware control.

5.1 DEVELOPMENT TOOLS AND TECHNOLOGIES

The system is implemented using a combination of software and hardware technologies:

Software:

- **Python** - Core programming and logic implementation
- **OpenCV** - Video capture and image processing
- **Dlib** - Face detection and landmark prediction
- **NumPy** - Mathematical computations
- **Firebase** - Cloud database for IoT integration

Hardware:

- **Webcam** - Captures real-time video input
- **ESP32 Microcontroller** - Handles IoT communication
- **DHT11 Sensor** - Measures temperature and humidity
- **Relay Module** - Controls humidifier
- **Humidifier** - Provides environmental adjustment

5.2 IMPLEMENTATION WORKFLOW

The system follows a continuous real-time workflow:

- System initializes and loads required models
- Webcam captures live video frames
- Face and eye regions are detected
- EAR is calculated to detect blinks
- Blink count and BPM are computed
- Eye condition is classified
- Data is stored locally and uploaded to Firebase
- If severe condition is detected, hardware response is triggered
- Results are displayed to the user

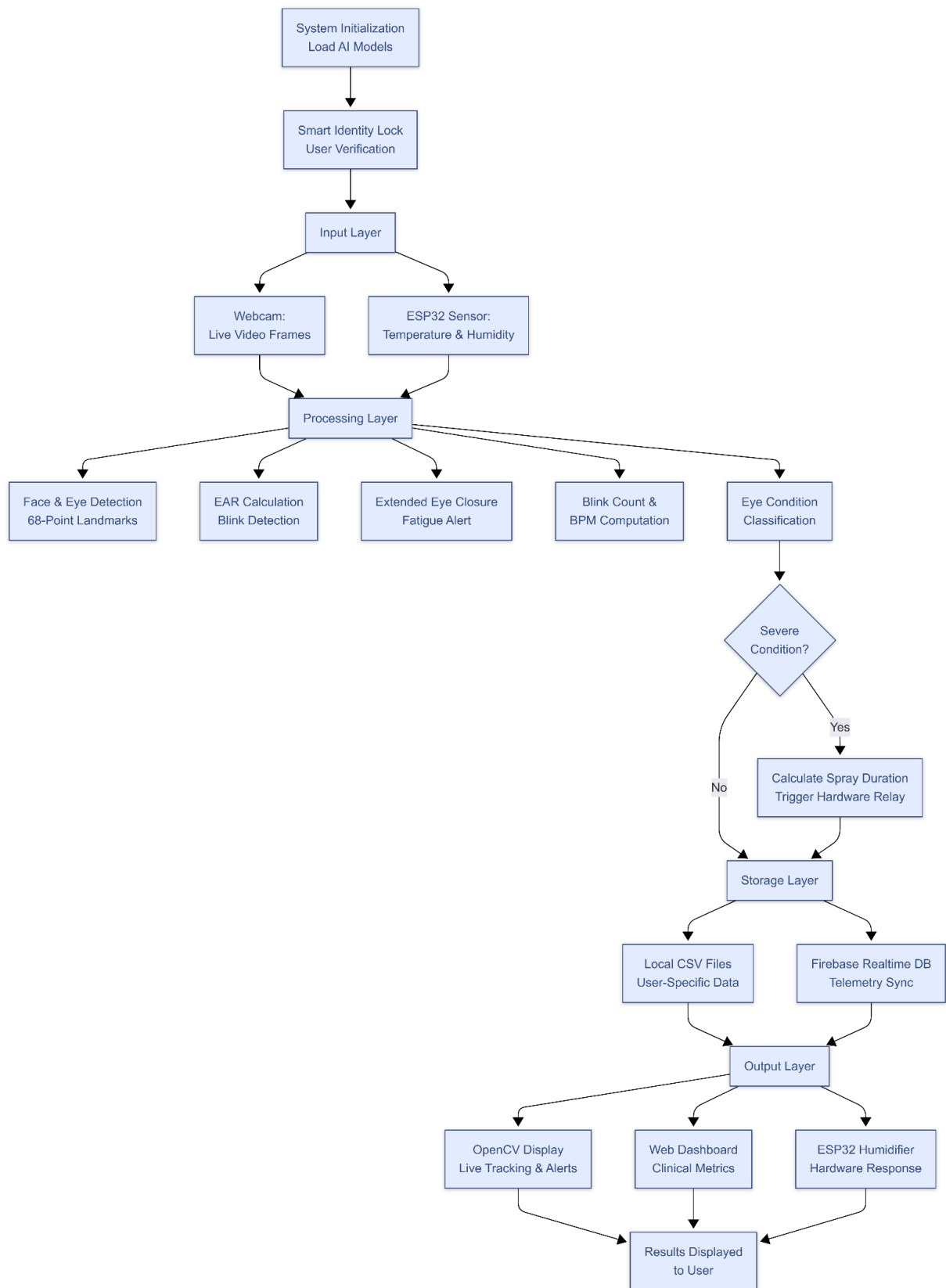


Figure 5.2 Workflow of System

5.3 IMPLEMENTATION BASED ON SYSTEM LAYERS

5.3.1 Input Layer Implementation

The webcam is initialized using the OpenCV library to capture continuous video frames in real time. The captured video stream is divided into individual frames, which act as input for further processing. The resolution is typically set to a standard size such as 640×480 to maintain a balance between processing speed and detection accuracy, ensuring smooth real-time performance.

User registration is implemented by capturing the user's facial image and generating a unique facial encoding. This encoding is stored as a numerical representation of facial features and is later used for real-time face recognition. This ensures that only registered users are monitored, improving system reliability and security.

Environmental data such as temperature and humidity is received from the ESP32 microcontroller through serial communication. The DHT11 sensor connected to ESP32 continuously measures environmental conditions and transmits them to the system, providing additional context for eye condition analysis.

This layer provides synchronized visual input and environmental data, forming the foundation for accurate real-time processing and analysis.

5.3.2 Processing Layer Implementation

Face detection is performed using dlib's frontal face detector, which identifies the location of the face within each video frame. The detection process converts frames into grayscale to reduce computational complexity and improve efficiency, as discussed in the methodology section.

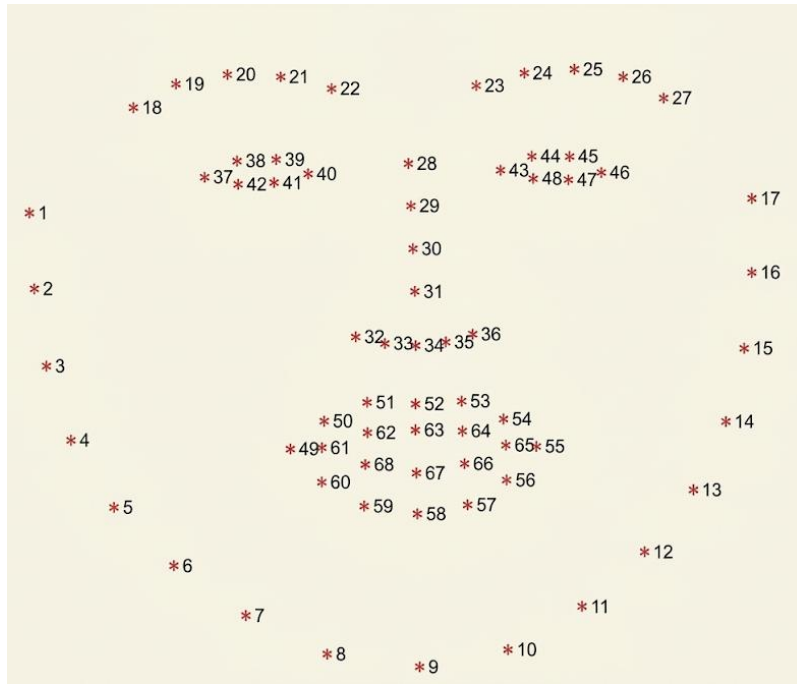


Figure 5.3.2.1 68-Point Facial Landmark Detection Model

Facial landmarks consisting of 68 specific points are extracted from the detected face using a pre-trained landmark predictor model. These points represent key facial regions such as eyes, nose, and mouth, enabling precise localization of eye regions.

The eye regions are isolated using specific landmark indices (points 37–48) and converted into array format using NumPy. This structured representation allows efficient mathematical computations for further analysis.

The Eye Aspect Ratio (EAR) is calculated using the geometric relationship between eye landmark points. The formula used is:

$$\text{EAR} = (V_1 + V_2) / (2 \times H)$$

where V_1 and V_2 represent vertical distances between eyelid points, and H represents the horizontal distance between eye corners



Figure 5.3.2.2 Open vs Closed Eye EAR Representation

Blink detection is implemented using a threshold-based approach combined with frame-count logic. When the EAR value falls below a predefined threshold (e.g., 0.23) for consecutive frames, the system identifies the eye as closed.

Blinks Per Minute (BPM) is calculated using the formula:

$$\text{BPM} = \text{Number of Blinks} / \text{Time (in minutes)}$$

This provides a quantitative measure of blinking frequency over time.

- If $\text{BPM} \geq 20 \rightarrow \text{Status} = \text{Healthy}$
- If $10 \leq \text{BPM} < 20 \rightarrow \text{Status} = \text{Moderate}$
- If $\text{BPM} < 10 \rightarrow \text{Status} = \text{Severe}$

Based on BPM values, the system classifies the eye condition into Healthy, Moderate, or Severe categories using predefined thresholds, enabling real-time health assessment.

After classification, the system displays the eye condition status on the screen as part of the user interface. To enhance clarity and usability, visual indicators such as colors are used:

- **Green color:** Represents Healthy condition
- **Yellow color:** Represents Moderate condition
- **Red color:** Represents Severe condition

Along with color indicators, the status text (Healthy, Moderate, Severe) is displayed on the video frame. This real-time visual feedback helps users easily understand their eye condition without

needing technical knowledge. This layer converts raw visual data into meaningful eye behaviour metrics using mathematical models and real-time analysis techniques.

5.3.3 Control & Hardware Interface Layer Implementation

Serial communication is established between the Python-based system and the ESP32 microcontroller using a standard baud rate such as 115200. This enables bidirectional data transfer between software and hardware components.

When a severe dry eye condition is detected (e.g., BPM below threshold), the system generates a control signal and transmits it to the ESP32. This signal acts as a trigger for initiating hardware response. The ESP32 activates a relay module, which controls the humidifier. The relay acts as an electronic switch, allowing safe operation of high-power devices while maintaining electrical isolation.

Time-proportional control is implemented to determine the duration of humidifier activation. The spray duration is calculated based on the severity level, ensuring efficient and controlled operation.

A cooldown mechanism is implemented to prevent repeated triggering within a short time interval (e.g., 20 seconds). This ensures safe operation and avoids over-humidification of the environment. This layer enables seamless integration between software analysis and physical hardware response, ensuring automated and safe corrective action.

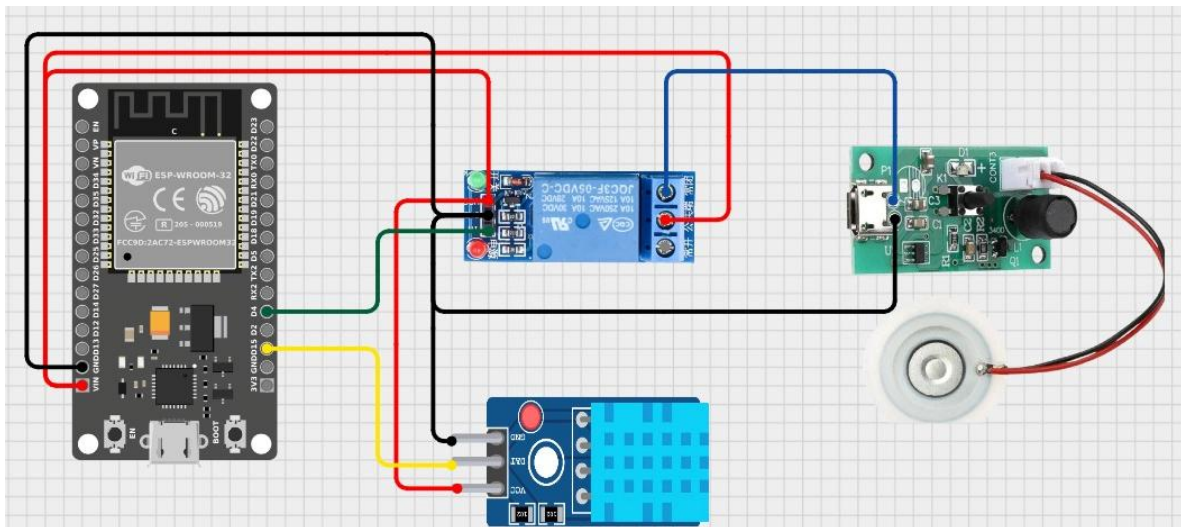


Figure 5.3.3.1 IoT Smart Humidifier Control System

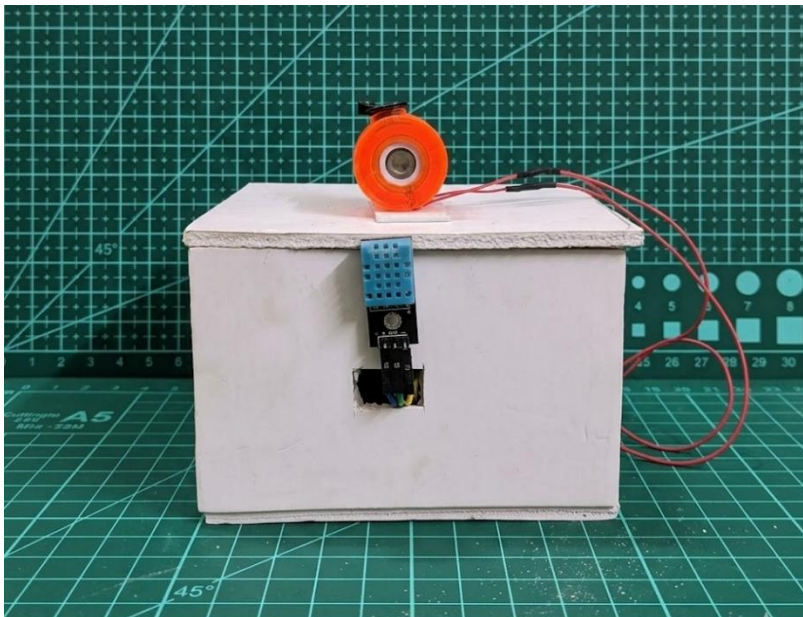


Figure 5.3.3.2 Physical Humidifier Setup (i)

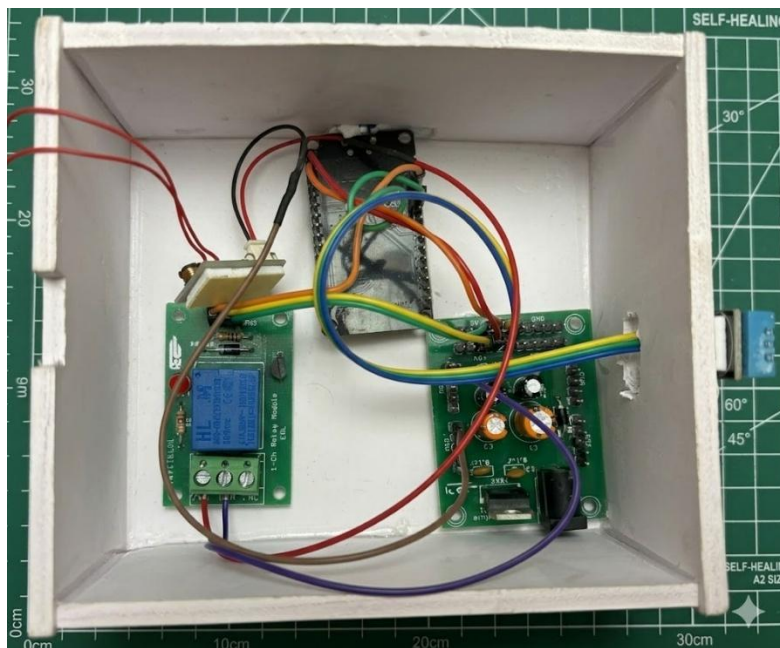


Figure 5.3.3.2 Physical Humidifier Setup (ii)

5.3.4 Data Management & IoT Layer Implementation

Session data is stored locally in CSV format, including blink count, BPM, eye condition, and timestamp. This structured format ensures easy data access and analysis. Blink count is recorded continuously as the number of detected blink events during a session. It serves as the primary metric for calculating BPM.

BPM values are computed dynamically and stored along with timestamps. Eye condition status (Healthy, Moderate, Severe) is also recorded for clear classification.

Firebase Realtime Database is integrated using the firebase admin library to enable cloud storage and real-time synchronization. Data is uploaded at regular intervals to ensure continuous monitoring.

Environmental data from ESP32 is also logged, and a web dashboard displays live BPM, environmental conditions, and system status. This supports remote monitoring and long-term analysis.

5.3.5 Output Layer Implementation

Real-time results are displayed on the system interface, including blink count, BPM, and eye condition status, allowing continuous monitoring of eye health. Blink count updates dynamically, while BPM values provide immediate feedback on blinking frequency. Eye condition status is shown using color-coded indicators, where green indicates healthy condition, yellow indicates moderate condition, and red indicates severe condition.

The web-based dashboard provides advanced diagnostics and hardware feedback, including the live status of the ESP32-controlled humidifier (ON/OFF). It tracks "Dry Eye Events" using strict thresholds, such as maintaining a severe BPM state for over 20 seconds, ensuring accurate detection of sustained dryness. The system also maintains a Patient Database using Facial Recognition, enabling access to past records and Matplotlib-generated trends for long-term analysis. Alerts are triggered during severe conditions, and graphs are generated using matplotlib to visualize blinking patterns over time,

where X-axis $\rightarrow$ Time and Y-axis $\rightarrow$ BPM,

helping identify trends and variations effectively.

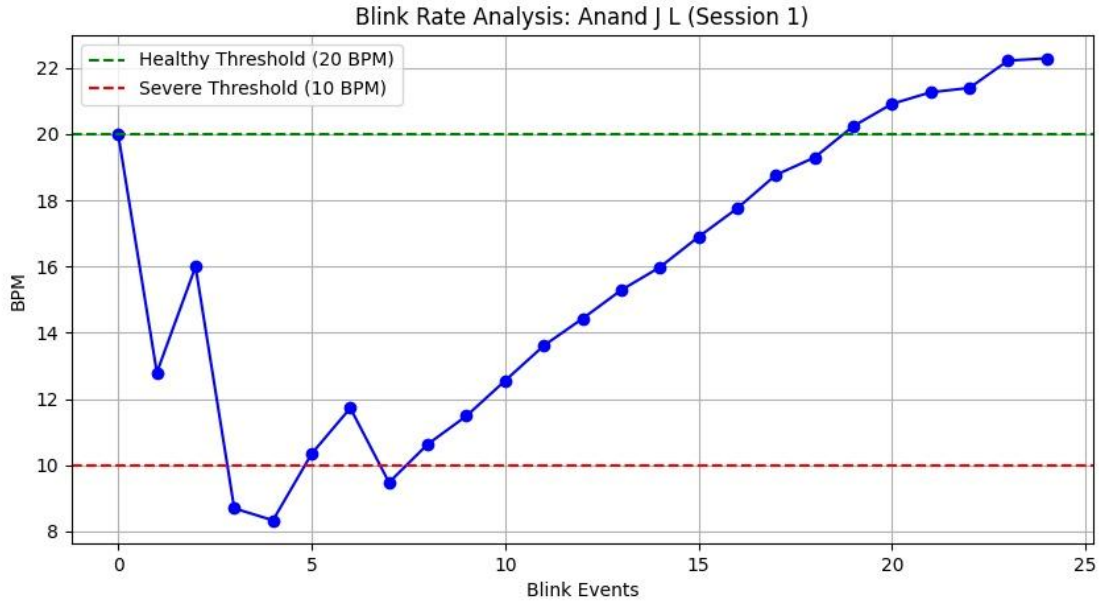


Figure 5.3.5 Blink Rate Trend Analysis with Health Thresholds

5.4 Performance Considerations

- **Real-Time Processing:** The system processes webcam input efficiently to ensure smooth, real-time operation. Optimized frame handling and reduced computational overhead help maintain consistent performance without lag.
- **Accuracy:** Blink detection is performed using the Eye Aspect Ratio (EAR) method with properly tuned thresholds. This ensures reliable identification of blinks while minimizing false detections.
- **Low Latency:** Communication between the software and ESP32 is optimized to reduce delays. This enables quick transmission of control signals for immediate hardware response.
- **Memory Efficiency:** The system manages memory by processing frames dynamically and storing only essential data. This prevents memory overflow and supports continuous operation.
- **System Stability:** Error handling and resource management techniques ensure stable long-term execution without crashes or interruptions.
- **Hardware Safety:** Controlled activation and cooldown mechanisms protect the humidifier from overuse, ensuring safe and reliable hardware operation.

CHAPTER-6

RESULTS AND ANALYSIS

The results and analysis of the Dry Eye Detection System are presented based on its design and prototype implementation. The evaluation focuses on how effectively the system achieves its objectives, particularly in terms of real-time performance, system responsiveness, and detection accuracy. As the project is currently in the development stage, the analysis is centered on observed outputs, system behavior, and expected performance under continuous operation. These observations highlight the potential of the proposed system in providing an efficient and automated solution for monitoring and managing dry eye conditions.

6.1 SYSTEM OUTPUT OBSERVATION

The system provides continuous real-time output through the interface, displaying:

- Live blink count
- Humidifier Status
- Eye condition status (Dry eye detection).
- Environmental parameters such as temperature and humidity.

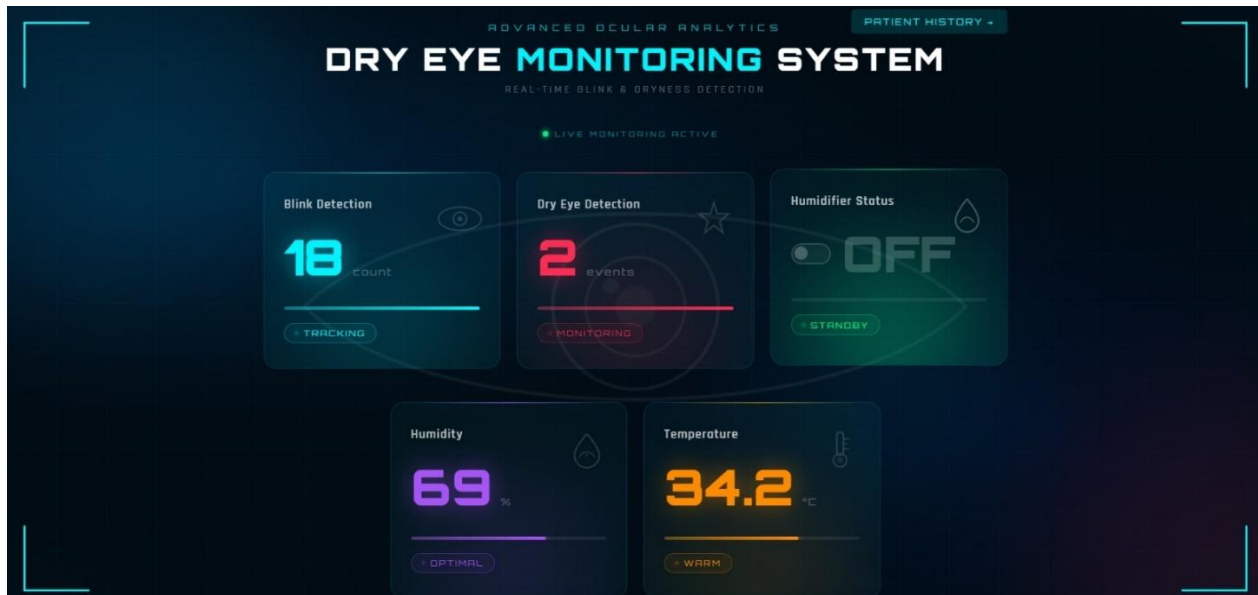


Figure 6.1.1: User Interface of the Dry Eye Monitoring System

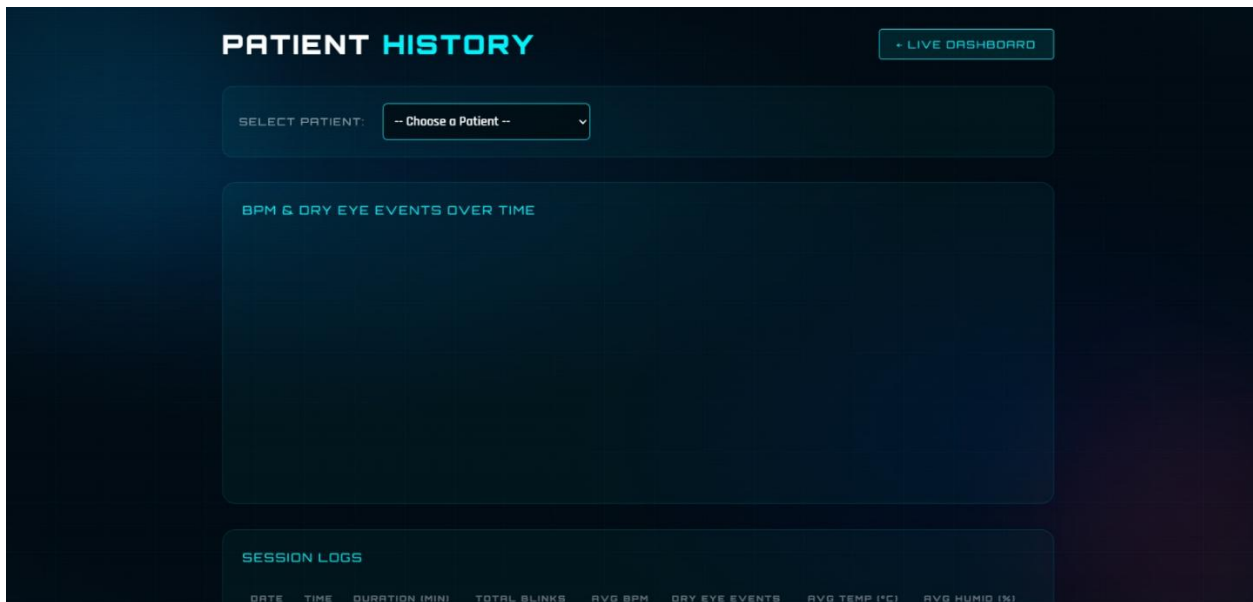


Figure 6.1.2: Patient History Interface (i)

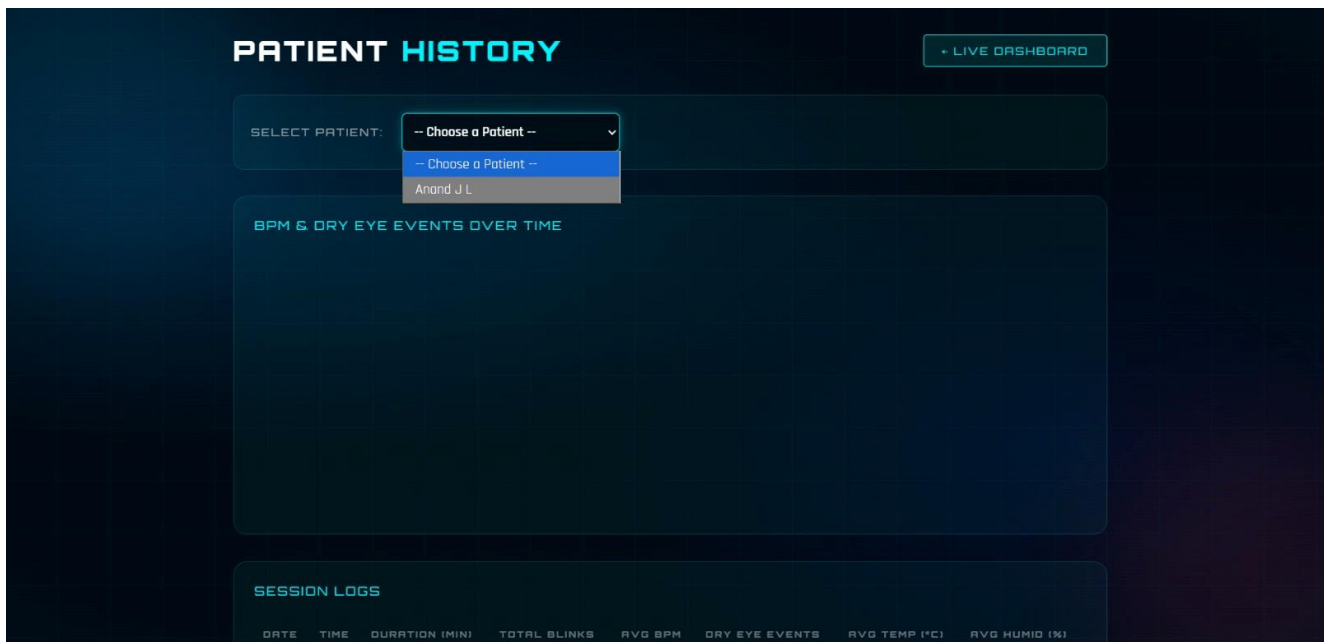


Figure 6.1.2: Patient History Interface (ii)



Figure 6.1.3: BPM & Dry eye events graph

6.2 ACCURACY OF BLINK DETECTION

The system demonstrates consistent performance in detecting eye blinks under normal operating conditions.

- Blink detection remains stable when the face is clearly visible
- EAR-based detection produces reliable results with minimal fluctuations
- False positives are limited when proper lighting is maintained

This confirms that the implemented approach is suitable for real-time monitoring.

6.3 SYSTEM RESPONSE AND AUTOMATION

The system reacts appropriately to changes in eye condition:

- When low blink rate is detected, the system transitions to severe status
- Automated triggering of the humidifier occurs without delay
- Cooldown mechanism ensures controlled hardware activation

This validates the effectiveness of the automated response mechanism.

6.4 DATA HANDLING AND MONITORING

The system successfully maintains and manages data during operation:

- Session data is recorded continuously
- Cloud synchronization enables remote access to system parameters
- Historical data supports basic trend observation

The separation of data management improves system organization and scalability.

6.5 FUNCTIONAL ANALYSIS

The system was analyzed based on its key functionalities:

FUNCTION	DESCRIPTION	STATUS
Real-Time Monitoring	Continuous tracking of eye activity	Implemented
Blink Detection	Detection using EAR calculation	Implemented
Eye Classification	Categorization of eye condition	Implemented
Automation Control	Hardware triggering based on condition	Implemented
Data Storage	Saving session data for analysis	Implemented
Cloud Integration	Remote data access via Firebase	Implemented
User Interface	Real-time display and alerts	Implemented

Table 6.5 Functional Modules and Implementation Status

6.6 PERFORMANCE ANALYSIS

PARAMETER	OBSERVATION
Response Time	Immediate output updates
Detection Accuracy	High under proper lighting
Processing Speed	Smooth real-time execution
System Stability	Continuous operation without failure
Hardware Response	Timely and reliable
Latency	Negligible delay observed
Resource Usage	Optimized and efficient

Table 6.6 Performance Analysis of the Proposed System

6.7 RESULTANT OUTCOMES

The implementation results in:

- Reliable detection of blinking patterns
- Real-time classification of eye condition
- Automated environmental adjustment
- Continuous data tracking for monitoring

6.8 LIMITATIONS

Despite effective performance, certain limitations were observed:

- Performance depends on lighting conditions
- Requires proper user positioning
- Limited adaptability to extreme variations (e.g., occlusions, glasses glare)
- Single-user operation only

6.9 SUMMARY

The system demonstrates stable real-time operation with accurate detection and effective automation. The results confirm that the proposed approach is practical and suitable for continuous eye monitoring applications.

CHAPTER-7

CONCLUSION AND FUTURE SCOPE

The developed system provides a functional solution for monitoring eye activity and identifying dry eye conditions using real-time video analysis. By combining blink detection with automated response, the system moves beyond passive observation and introduces active intervention.

The integration of hardware control and cloud-based data handling enhances the system's usability and extends its capabilities beyond a standalone application. The implementation confirms that such systems can operate reliably with minimal user interaction.

Overall, the project demonstrates a practical approach to combining computer vision and IoT for health-related monitoring.

7.1 CONCLUSION

The developed Dry Eye Detection System successfully demonstrates the integration of real-time computer vision and automated response mechanisms for monitoring eye activity. The system effectively captures live video, analyses blink patterns, and classifies eye conditions with consistent performance.

The implementation confirms that EAR-based blink detection can be reliably used for continuous eye monitoring. The system responds promptly to changes in blink rate and ensures timely activation of the humidifier, maintaining a balance between functionality and safety through controlled operation.

The inclusion of data management and cloud integration enhances the overall capability of the system by enabling remote monitoring and maintaining historical records. This makes the system more scalable and suitable for extended applications beyond a standalone setup.

In conclusion, the project provides a practical and efficient solution for detecting dry eye conditions, combining accuracy, automation, and usability. The system lays a strong foundation for future improvements and real-world deployment in health monitoring applications.

7.2 FUTURE SCOPE

The Dry Eye Detection System can be further enhanced with additional features and improvements to increase its effectiveness and usability. Some possible future developments include:

- **Improved Detection Accuracy:** Enhancing performance under low light conditions and reducing errors caused by occlusions or reflections
- **Personalized Calibration:** Adapting the system to individual user blink patterns for more accurate classification
- **Mobile Application Development:** Creating a portable version of the system for smartphones and tablets
- **Advanced Analytics:** Incorporating machine learning for predictive analysis and early detection of eye conditions
- **Wearable Integration:** Connecting the system with smart glasses or wearable devices for continuous monitoring
- **Enhanced User Interface:** Developing a more interactive dashboard with detailed visualizations and insights
- **Multi-User Support:** Extending the system to monitor multiple users simultaneously
- **Smart Alert System:** Implementing notifications and reminders to encourage healthy blinking habits
- **Cloud Expansion:** Improving cloud capabilities for long-term data storage and advanced remote monitoring
- **Healthcare Integration:** Linking the system with medical platforms for professional diagnosis and consultation

REFERENCES

- [1] Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation. <https://doi.org/10.1162/neco.1997>
- [2] Viola, P., & Jones, M. J. (2004). *Robust Real-Time Face Detection*. International Journal of Computer Vision. <https://doi.org/10.1023/B:VISI.0000013087>
- [3] Lee, W. B., et al. (2010). *Eye Blinking Detection Using SVM and AdaBoost*. Proceedings of the International Conference on Control, Automation and Systems.
- [4] Portello, J. K., et al. (2013). *Effect of Blink Rate and Completeness on Dry Eye Signs and Symptoms in Contact Lens Wearers*. Investigative Ophthalmology & Visual Science. <https://doi.org/10.1167/iovs.13>
- [5] Kulkarni, U. P., et al. (2013). *Drowsiness Detection Based on Eye Shape Measurement*. International Conference on Electrical, Electronics, and Mechanical Engineering.
- [6] Sugur, S. M., et al. (2013). *Drowsiness Detection Based on Eye Shape Measurement*. International Conference on Electrical, Electronics, and Mechanical Engineering.
- [7] Marcos-Ramiro, I., et al. (2014). *Real-Time Eye Blink Detection and Tracking Using Randomized Forest*. Proceedings of the International Conference on Pattern Recognition (ICPR).
- [8] Donahue, J., et al. (2015). *Long-Term Recurrent Convolutional Networks for Visual Recognition and Description*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). <https://doi.org/10.1109/CVPR.2015>
- [9] Soukupová, E., & Čech, J. (2016). *Real-Time Eye Blink Detection Using Facial Landmarks*. Proceedings of the Computer Vision Slovak Conference (SMVK).
- [10] Anas, M. M., et al. (2017). *Online Eye Status Detection in the Wild with Convolutional Neural Networks*. Proceedings of the International Conference on Computational Science and Its Applications (ICCSA).
- [11] Fogelton, T., & Benešová, K. (2017). *Blink Detection and Identification of Complete and Incomplete Blinks*. Computer Vision Slovak Conference (SMVK).

- [12] Fogelton, T., & Benešová, K. (2018). *Blink Classification: Complete and Incomplete Blinks Detection Using Recurrent Neural Network*. Pattern Recognition Letters. <https://doi.org/10.1016/j.patrec.2018>
- [13] Hu, S., et al. (2020). *Towards Real-Time Eyeblink Detection in the Wild: Dataset, Theory and Practices*. IEEE Transactions on Image Processing. <https://doi.org/10.1109/TIP.2020>
- [14] Cortacero, S., et al. (2020). *RT-BENE: A Dataset for Eye State Detection in Real-Time*. Pattern Recognition Letters. <https://doi.org/10.1016/j.patrec.2020>
- [15] Jordan, J., et al. (2020). *Eye-LRCN: A Long-Term Recurrent Convolutional Network for Eye Blink Completeness Detection*. IEEE Transactions on Cybernetics. <https://doi.org/10.1109/TCYB.2020>
- [16] Shu, Y., et al. (2022). *An Improved Denoising Method for Eye Blink Detection Using Millimeter Wave Radar*. Springer. <https://doi.org/10.1186/s13634-022-00841-y>
- [17] Zeng, B., et al. (2023). *Real-Time Multi-Person Eyeblink Detection in the Wild for Untrimmed Video*. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). <https://doi.org/10.1109/CVPR.2023>
- [18] Shahbakhti, M., et al. (2023). *Fusion of EEG and Eye Blink Analysis for Detection of Driver Fatigue*. IEEE Transactions on Neural Systems and Rehabilitation Engineering. <https://doi.org/10.1109/TNSRE.2023>
- [19] de la Cruz, G., et al. (2024). *Eye-LRCN: A Long-Term Recurrent Convolutional Network for Eye Blink Completeness Detection*. IEEE Transactions on Neural Networks and Learning Systems. <https://doi.org/10.1109/TNNLS.2024>
- [20] Deepu, S. S., et al. (2024). *EEG-Based Eye Blink Detection and Interpretation for Human-Computer Interaction*. Proceedings of the IEEE International Conference on Brain-Computer Interface and Human-Technology Interaction (iCon-BCIHT). <https://doi.org/10.1109/iCon-BCIHT.2024>
- [21] Hong, S., et al. (2024). *Robust Eye Blink Detection Using Dual Embedding Video Vision Transformer*. Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV). <https://doi.org/10.1109/WACV.2024>

APPENDIX

APPENDIX A: SYSTEM THRESHOLDS AND CLASSIFICATION CRITERIA

The following are the core mathematical thresholds and clinical criteria implemented in the Dry Eye Detection system:

A.1 Blinks Per Minute (BPM) Classification:

- Healthy Condition: $\text{BPM} \geq 20$
- Moderate Condition: $10 \leq \text{BPM} < 20$
- Severe Condition: $\text{BPM} < 10$

A.2 Eye Aspect Ratio (EAR) Thresholds:

- Blink Detection Threshold: $\text{EAR} < 0.27$
- Fast Blink Registration: Minimum 1 consecutive frame
- Fatigue / Microsleep Alert: Minimum 15 consecutive frames

A.3 Hardware Automation Triggers:

- Humidifier Activation: Triggered exclusively when eye condition reaches "Severe" status.
- Dry Eye Event Timer: Triggered after 20 continuous seconds of low-BPM tracking.
- Relay Cooldown Period: 20 seconds between consecutive sprays.

APPENDIX B: LIST OF HARDWARE AND SENSOR COMPONENTS

The system utilizes the following physical components for data acquisition and automated response:

- Processing Unit: PC/Laptop for running Python computer vision scripts.
- Vision Input: Standard Webcam to capture live video frames.
- Edge Processing: ESP32 Microcontroller to handle physical IoT control.
- Environmental Sensor: DHT11 Sensor to capture ambient room temperature and humidity.
- Actuator Unit: 5V Electronic Relay Module to safely isolate the PC from humidifier voltage.
- Correction Mechanism: Electronic Room Humidifier for environmental adjustment.

APPENDIX C: SOFTWARE AND TOOLS USED

The following software libraries, frameworks, and tools were used in the development of the system:

- Python: Core programming and logic implementation.
- OpenCV (cv2): Video capture and grayscale image processing.
- Dlib: 68-point facial detection and landmark prediction.
- face_recognition: Smart Identity Lock and user verification.
- NumPy: Mathematical computations and array structuring.
- Firebase Realtime Database: Cloud database for IoT integration and telemetry sync.
- Matplotlib: Generating graphical visualizations of blink patterns over time.
- Web Technologies: HTML5, CSS3, and JavaScript for the clinical dashboard.

APPENDIX D: APPLICATION FEATURES

- Real-Time Monitoring: Continuously tracks user eye activity via standard webcam.
- Smart Identity Lock: Verifies user identity to prevent imposter data corruption.
- Blink Detection: Accurately detects eye blinks using the EAR mathematical model.
- Eye Condition Classification: Correctly categorizes eye condition into Healthy, Moderate, and Severe states.
- Automation: Automatically triggers the humidifier relay based on system analysis.
- Data Handling: Stores and manages session data effectively in local CSV files.
- Cloud Connectivity: Enables remote monitoring through IoT Firebase integration.
- User Interaction: Provides clear and understandable feedback via OpenCV overlays and the web dashboard.

APPENDIX E: COMPREHENSIVE SOURCE CODE IMPLEMENTATION

E.1 Smart Identity Lock & Data Protection Module

To ensure clinical data logged to the local CSV and Firebase database is entirely accurate, this feature uses deep learning facial encodings to verify the user at the workstation. It prevents imposter data corruption and dynamically routes data to patient-specific files.

```

1  # 1. SMART IDENTITY LOCK
2  if time.time() - last_id_check > 2.0:
3      if main_rect is not None:
4          face_locs = face_recognition.face_locations(rgb)
5          if face_locs:
6              largest_loc = max(face_locs, key=lambda loc: (loc[2]-loc[0]) * (loc[1]-loc[3]))
7              face_encs = face_recognition.face_encodings(rgb, [largest_loc])
8
9          if face_encs:
10             matches = face_recognition.compare_faces(known_encs, face_encs[0], 0.6)
11             if True in matches:
12                 match_name = known_names[matches.index(True)]
13                 if match_name != current_user:
14                     # -- UPLOAD HISTORY BEFORE SWITCHING USERS --
15                     upload_session_history(current_user, session_id, total_blinks, session_start, dry_eye_events, current_temp, current_humidity)
16                     save_session_graph(current_user, session_id, bpm_history)
17
18                     current_user = match_name
19                     total_blinks = 0
20                     dry_eye_events = 0
21                     session_start = time.time()
22                     last_boost_time = 0
23                     bpm_history = []
24                     fatigue_alert = False
25                     csv_path = os.path.join(DATA_DIR, f"{current_user}.csv")
26
27                     intensity_path = os.path.join(DATA_DIR, f"{current_user}_intensity.txt")
28                     if os.path.exists(intensity_path):
29                         with open(intensity_path, 'r') as f:
30                             try: humidifier_intensity = int(f.read().strip())
31                             except: humidifier_intensity = 50
32                     else: humidifier_intensity = 50

```

Figure E.1.1 Smart Identity Lock & Data Protection Module Code

```

1      if not os.path.exists(csv_path):
2          session_id = 1
3          with open(csv_path, 'w', newline='') as f:
4              csv.writer(f).writerow(["Session_ID", "Timestamp", "Total_Blinks", "BPM", "Status"])
5      else:
6          with open(csv_path, 'r') as f:
7              reader = list(csv.reader(f))
8              session_id = int(reader[-1][0]) + 1 if len(reader) > 1 else 1
9
10     if current_user not in ["Scanning...", "Unregistered"]:
11         print("\n[WARN] Unregistered person detected! Pausing active session.")
12         upload_session_history(current_user, session_id, total_blinks, session_start, dry_eye_events, current_temp, current_humidity)
13         save_session_graph(current_user, session_id, bpm_history)
14         current_user = "Unregistered"
15         session_start = None
16         csv_path = None
17         fatigue_alert = False
18         last_boost_time = 0
19
20     else:
21         if current_user == "Scanning...":
22             print("[WARN] No face detected. Searching...", end="\r")
23     last_id_check = time.time()

```

Figure E.1.2 Smart Identity Lock & Data Protection Module Code

E.2 The Computer Vision & AI Diagnostic Pipeline

This core module handles the real-time video stream, isolates the 68-point facial landmarks, and continuously calculates the Eye Aspect Ratio (EAR). It applies calibrated thresholds to count natural blinks and trigger severe fatigue warnings.

```
1 # 2. BLINK & FATIGUE TRACKING
2 if main_rect is not None and current_user not in ["Scanning...", "Unregistered"]:
3     shape = predictor(gray, main_rect)
4     shape = face_utils.shape_to_np(shape)
5     ear = (eye_aspect_ratio(shape[36:42]) + eye_aspect_ratio(shape[42:48])) / 2.0
6
7     if ear < 0.23:
8         COUNTER += 1
9         if COUNTER >= FATIGUE_FRAMES: fatigue_alert = True
10    else:
11        if COUNTER >= 2:
12            total_blinks += 1
13            if csv_path and session_start:
14                bpm_history.append(bpm)
15                with open(csv_path, 'a', newline='') as f:
16                    csv.writer(f).writerow([session_id, datetime.now().strftime("%H:%M:%S"), total_blinks, round(bpm, 1), status_text])
17            COUNTER = 0
18            fatigue_alert = False
```

Figure E.2 The Computer Vision & AI Diagnostic Pipeline Code

E.3 Hardware Automation & Cloud IoT Synchronization

This block represents the Control and Data Management Layers. It triggers the physical ESP32 humidifier via serial communication and compiles the entire telemetry state into a JSON payload for cloud synchronization.

```

1
2 # 3. BPM & STATUS MATH
3 status_color = (0, 0, 0)
4 if session_start and current_user not in ["Scanning...", "Unregistered"]:
5     elapsed = time.time() - session_start
6     dur_min = elapsed / 60
7     bpm = total_blinks / dur_min if dur_min > 0.02 else 20.0
8
9     if bpm >= 20:
10         status_text, status_color = "Healthy", (0, 255, 0)
11         last_boost_time = 0
12     elif 10 <= bpm < 20:
13         status_text, status_color = "Moderate", (0, 255, 255)
14         last_boost_time = 0
15     else:
16         status_text, status_color = "Severe", (0, 0, 255)
17
18     # --- AUTO BOOST LOGIC ---
19     if elapsed > 20 and (time.time() - last_boost_time > 20.0):
20         dry_eye_events += 1 # Increase dry eye event count for dashboard
21
22         humidifier_intensity = min(100, humidifier_intensity + 10)
23         with open(os.path.join(DATA_DIR, f"{current_user}_intensity.txt"), 'w') as f:
24             f.write(str(humidifier_intensity))
25
26         duration_seconds = int(humidifier_intensity / 10)
27         if arduino is not None:
28             try: arduino.write(f"{duration_seconds}\n".encode())
29             except: pass
30
31         print(f"\n[AUTO] Status is Severe! Spraying for {duration_seconds}s.")
32         last_boost_time = time.time()
33

```

Figure E.3.1 Hardware Automation & Cloud IoT Synchronization Code

```

1 # --- FIREBASE LIVE UPDATE FOR HTML DASHBOARD (Every 3 seconds) ---
2 if firebase_enabled and (time.time() - last_firebase_update > 3.0):
3     try:
4         # Pushing exactly to the branch expected by index.html
5         daily_ref = db.reference("dryeyescm")
6         daily_ref.update({
7             "blinkCount": total_blinks,
8             "dryeyeDetection": dry_eye_events,
9             "humidifireStatus": 1 if status_text == "Severe" else 0,
10            "temperature": current_temp,
11            "humidity": current_humidity
12        })
13        last_firebase_update = time.time()
14    except Exception as e:
15        print(f"[FIREBASE UPLOAD ERROR] {e}")
16 else:
17     bpm = 0.0
18     status_text = "Paused" if current_user == "Unregistered" else "Scanning"
19     status_color = (0, 0, 255) if current_user == "Unregistered" else (0, 0, 0)
20

```

Figure E.3.2 Hardware Automation & Cloud IoT Synchronization Code

E.4 Real-Time Asynchronous Web Dashboard Engine

This JavaScript implementation acts as the presentation layer. It bypasses the need for manual page reloads by establishing an active listener (onValue) to the Firebase Realtime Database. It instantly catches state changes and dynamically manipulates the DOM elements.

```
1 import { initializeApp } from "https://www.gstatic.com/firebasejs/10.12.2/firebase-app.js";
2 import { getDatabase, ref, onValue } from "https://www.gstatic.com/firebasejs/10.12.2/firebase-database.js";
3
4 // 1. Initialize Firebase Connection
5 const firebaseConfig = {
6   apiKey: "AIzaSyCSft5LizM6QqTbqVjXrcqUirGhLxRl68",
7   authDomain: "dry-eyes-detection.firebaseio.com",
8   databaseURL: "https://dry-eyes-detection-default-rtdb.asia-southeast1.firebaseio.com",
9   projectId: "dry-eyes-detection",
10  storageBucket: "dry-eyes-detection.firebaseio.com",
11  messagingSenderId: "182409923761",
12  appId: "1:182409923761:web:d5852cdac2d25e6af3a690",
13  measurementId: "G-Z44F9W21P0"
14 };
15
16 const app = initializeApp(firebaseConfig);
17 const db = getDatabase(app);
18
19 let blinkMax = 1;
20 let dryMax = 1;
21
22 function popAnim(el) {
23   el.classList.remove('pop');
24   void el.offsetWidth;
25   el.classList.add('pop');
26 }
```

Figure E.4.1 Real-Time Asynchronous Web Dashboard Engine Code

```
1 // 2. Live Blink Tracking Listener
2
3 onValue(ref(db, "dryeyescom/blinkCount"), (snapshot) => {
4   const value = snapshot.val() ?? 0;
5   const el = document.getElementById("blink");
6   el.textContent = value;
7   popAnim(el);
8   if (value > blinkMax) blinkMax = value;
9   document.getElementById("blinkBar").style.width = Math.min((value / blinkMax) * 100, 100) + "%";
10 });
11
12 // 3. Dry Eye Event Listener & UI Alert Trigger
13
14 let previousDryCount = null; // Track old value
15
16 onValue(ref(db, "dryeyescom/dryeyeDetection"), (snapshot) => {
17   const value = snapshot.val() ?? 0;
18   const el = document.getElementById("dry");
19   el.textContent = value;
20   popAnim(el);
21   if (value > dryMax) dryMax = value;
22   document.getElementById("dryBar").style.width = Math.min((value / dryMax) * 100, 100) + "%";
23
24   // Show alert popup ONLY if the count has actually increased!
25   if (previousDryCount !== null && value > previousDryCount) {
26     const alertBox = document.getElementById("alertBox");
27     alertBox.classList.add("show");
28     alertBox.classList.remove("shake");
29     void alertBox.offsetWidth;
30     alertBox.classList.add("shake");
31     setTimeout(() => alertBox.classList.remove("show"), 3000);
32   }
33
34   previousDryCount = value; // Save the new number to compare against next time
35 });
36
```

Figure E.4.2 Real-Time Asynchronous Web Dashboard Engine Code

```

1 // 4. Hardware Humidifier Status Listener
2
3 /* Humidifier - 0 = OFF, 1 = ON */
4 onValue(ref(db, "dryeyescm/humidifierStatus"), (snapshot) => {
5   const value = snapshot.val() ?? 0;
6   const isOn = value === 1;
7   document.getElementById("humidSwitch").classList.toggle("is-on", isOn);
8   const word = document.getElementById("humidWord");
9   word.textContent = isOn ? "ON" : "OFF";
10  word.className = "toggle-word " + (isOn ? "on" : "off");
11  document.getElementById("humidBar").style.width = isOn ? "100%" : "0%";
12  document.getElementById("humidStatus").textContent = isOn ? "Active" : "Standby";
13 });
14
15 // 5. Hardware Humidity Listener
16
17 /* Humidity - 0-100% */
18 onValue(ref(db, "dryeyescm/humidity"), (snapshot) => {
19   const value = snapshot.val() ?? null;
20   const el = document.getElementById("humValue");
21   if (value === null) { el.textContent = "--"; return; }
22   el.textContent = Math.round(value);
23   popAnim(el);
24   document.getElementById("humBar").style.width = Math.min(value, 100) + "%";
25
26   const status = document.getElementById("humStatus");
27   if (value < 30) status.textContent = "Low";
28   else if (value > 70) status.textContent = "High";
29   else status.textContent = "Optimal";
30 });

```

Figure E.4.3 Real-Time Asynchronous Web Dashboard Engine Code

```

1 // 6. Hardware Temperature Listener
2
3 /* Temperature - assumed range 0-50°C for bar */
4 onValue(ref(db, "dryeyescm/temperature"), (snapshot) => {
5   const value = snapshot.val() ?? null;
6   const el = document.getElementById("tempValue");
7   if (value === null) { el.textContent = "--"; return; }
8   el.textContent = parseFloat(value).toFixed(1);
9   popAnim(el);
10  const pct = Math.min(Math.max(((value - 0) / 50) * 100, 0), 100);
11  document.getElementById("tempBar").style.width = pct + "%";
12
13  const status = document.getElementById("tempStatus");
14  if (value < 18) status.textContent = "Cold";
15  else if (value > 28) status.textContent = "Warm";
16  else status.textContent = "Comfortable";
17 });

```

Figure E.4.4 Real-Time Asynchronous Web Dashboard Engine Code

E.5 Historical Patient Data Visualization Engine

This feature operates on the Patient History dashboard, utilizing the Chart.js library to generate dynamic, dual-axis medical graphs. It plots a patient's historical Average BPM against isolated Dry Eye Events over time, allowing for longitudinal clinical analysis.

```
1 // 3. Render the Chart.js Graph
2 function updateGraph(userName, labels, bpmData, dryEventsData) {
3   const ctx = document.getElementById('historyChart').getContext('2d');
4
5   if(myChart) myChart.destroy(); // Destroy old graph before making a new one
6
7   myChart = new Chart(ctx, {
8     type: 'line',
9     data: {
10      labels: labels,
11      datasets: [
12        {
13          label: 'Average BPM',
14          data: bpmData,
15          borderColor: '#00f5ff',
16          backgroundColor: 'rgba(0, 245, 255, 0.1)',
17          tension: 0.3,
18          fill: true
19        },
20        {
21          label: 'Dry Eye Events',
22          data: dryEventsData,
23          borderColor: '#ff2d55',
24          backgroundColor: 'rgba(255, 45, 85, 0.1)',
25          tension: 0.3,
26          fill: true,
27          yAxisID: 'y1' // Put this on a separate axis scale
28        }
29      ]
30    },
```

Figure E.5.1 Historical Patient Data Visualization Engine Code

```
1   options: {
2     responsive: true,
3     maintainAspectRatio: false,
4     plugins: {
5       legend: { labels: { color: 'white', font: { family: 'Rajdhani' } } }
6     },
7     scales: {
8       x: { ticks: { color: 'rgba(255,255,255,0.5)' }, grid: { color: 'rgba(255,255,255,0.05)' } },
9       y: {
10        title: { display: true, text: 'Blinks per Min (BPM)', color: '#00f5ff' },
11        ticks: { color: '#00f5ff' }, grid: { color: 'rgba(255,255,255,0.05)' }
12      },
13       y1: {
14        position: 'right',
15        title: { display: true, text: 'Dry Eye Events', color: '#ff2d55' },
16        // FIX: Added precision: 0 to force whole numbers only
17        ticks: { color: '#ff2d55', stepSize: 1, precision: 0 },
18        grid: { drawOnChartArea: false }
19      }
20     }
21   }
22 });
23 }
```

Figure E.5.2 Historical Patient Data Visualization Engine Code